

SQL Code

```
1  -- =====
2  -- THE UNLOX ACADEMY - Weekly Assessment 3 - SQL Foundations
3  -- Name: Shoyal Halder
4  -- Dataset: flipcart.products
5  -- Date : 08-07-2026
6  -- =====
7
8
9  -- =====
10 -- SECTION A - THEORY (write answer letter a/b/c/d)
11 -- =====
12
13 -- A1. Schema vs database relationship in MySQL
14 -- Answer: c) Schema and database are the same thing in MySQL
15
16 -- A2. TRUNCATE TABLE belongs to which command family
17 -- Answer: b) DDL (Data Definition Language)
18
19 -- A3. Best data type for storing prices (e.g. 299.50, 79999.00)
20 -- Answer: b) DECIMAL
21
22 -- A4. INSERT with non-existent customer_id under FK constraint
23 -- Answer: c) The INSERT fails with a foreign key constraint error
24
25 -- A5. WHERE vs HAVING
26 -- Answer: b) WHERE filters individual rows before aggregation; HAVING filters groups
  after aggregation
27
28 -- A6. SELECT * FROM products WHERE avg_rating = NULL;
29 -- Answer: c) Returns 0 rows
30
31 -- A7. COUNT(*) vs COUNT(column_name)
32 -- Answer: b) COUNT(*) counts all rows; COUNT(column_name) counts only non-NULL
  values in that column
33
34 -- A8. Command that drops table structure + data, no ROLLBACK
35 -- Answer: c) DROP TABLE table_name
36
37
38 -- =====
39 -- SECTION B - OUTPUT PREDICTION (describe row count / values)
40 -- =====
41
42 -- B1. SELECT COUNT(*) FROM products WHERE category = 'Electronics';
43 -- Answer: Returns a single row with a single column name count(*), value = 8.
44
45 -- B2. SELECT * FROM products WHERE price BETWEEN 1000 AND 3000;
46 -- Answer: Returns 11 rows, all 10 columns, where price is between ₹1000 and ₹3000
  inclusive.
47
```

```

48 -- B3. SELECT product_name FROM products WHERE category = 'Books' AND price < 400
    ORDER BY price DESC LIMIT 1;
49 -- Answer: Returns 1 row – product_name = 'The Silent Patient' (the most expensive
    Book under ₹400).
50
51 -- B4. SELECT * FROM products WHERE avg_rating IS NULL;
52 -- Answer: Returns 3 rows, all 10 columns, where avg_rating IS NULL.
53
54 -- B5. SELECT MAX(price) FROM products WHERE category = 'Books';
55 -- Answer: Returns a single value = 499.00 (the highest price among Books).
56
57 -- B6. SELECT category, COUNT(*) FROM products WHERE is_active = TRUE GROUP BY
    category HAVING COUNT(*) > 4;
58 -- Answer: Returns 3 rows – Electronics (7), Apparel (6), Home (5) – categories with
    more than 4 active products.
59
60 -- B7. CASE WHEN price < 500 THEN 'Budget' ... query on Beauty category
61 -- Answer: Returns 4 rows (all Beauty products), each with product_name and a computed
    tier column: Nykaa Matte Lipstick → Mid, Lakme Eye Liner → Budget, Mamaearth Face Wash
    → Budget, WOW Skin Vitamin C Serum → Mid.
62
63 -- B8. SELECT product_name, COALESCE(avg_rating, 0) AS rating FROM products WHERE
    stock_quantity = 0;
64 -- Answer: Returns 2 rows – JBL Flip 6 Speaker (4.30) and WOW Skin Vitamin C Serum
    (4.50) – both out-of-stock products, neither has a NULL rating so COALESCE had nothing
    to substitute.
65
66 -- =====
67 -- SECTION C - APPLIED SQL (write actual query, format properly)
68 -- =====
69
70 -- --- C1: Basic SELECT + WHERE + ORDER BY ---
71
72 -- C1. Display all products, all columns.
73 SELECT * FROM products;
74
75 -- C2. product_name and price of all Books.
76 SELECT product_name, price FROM products WHERE category = 'Books';
77
78 -- C3. Products priced above 10,000, sorted highest to lowest.
79 SELECT * FROM products WHERE price > 10000.00 ORDER BY price DESC;
80
81 -- C4. Top 5 most expensive Electronics products (name + price).
82 SELECT product_name, price FROM products WHERE category = 'Electronics' ORDER BY price
    DESC LIMIT 5;
83
84 -- --- C2: Filtering Variations (IN, BETWEEN, LIKE) ---
85
86 -- C5. Products in Electronics or Apparel. Use IN.
87 SELECT * FROM products WHERE category IN ('Electronics', 'Apparel');
88
89 -- C6. Products priced between 500 and 2000 (inclusive).
90 SELECT * FROM products WHERE price BETWEEN 500 AND 2000;
91

```

```

92  -- C7. Products whose name contains 'Watch'.
93  SELECT * FROM products WHERE product_name LIKE '%Watch%';
94
95  -- C8. Products whose brand starts with 'S'.
96  SELECT * FROM products WHERE brand LIKE 'S%';
97
98  -- --- C3: DISTINCT + Aggregate Functions ---
99
100 -- C9. List unique categories.
101 SELECT DISTINCT category FROM products;
102
103 -- C10. Total number of products in catalogue.
104 SELECT COUNT(*) AS total_no_products FROM products;
105
106 -- C11. Average price of all Books.
107 SELECT AVG(price) AS avg_book_price FROM products WHERE category = 'Books';
108
109 -- C12. Max and min price across all products, single query.
110 SELECT MAX(price) , MIN(price) FROM products;
111
112 -- --- C4: GROUP BY ---
113
114 -- C13. Count of products per category.
115 SELECT category, COUNT(*) AS product_count FROM products GROUP BY category;
116
117 -- C14. Total stock quantity per category.
118 SELECT category, SUM(stock_quantity) AS total_stock FROM products GROUP BY category;
119
120 -- C15. Average price per category, sorted highest to lowest.
121 SELECT category, AVG(price) AS avg_price FROM products GROUP BY category ORDER BY
avg_price DESC;
122
123 -- C16. Count of products + avg price per brand, only brands with more than 1 product.
124 SELECT brand, COUNT(*) AS product_count , AVG(price) AS avg_price FROM products GROUP
BY brand HAVING COUNT(*) > 1;
125 -- returns 0 rows, no brand has more than 1 product in this dataset
126
127 -- --- C5: HAVING + LIMIT ---
128
129 -- C17. Categories with more than 4 active products.
130 SELECT category, COUNT(*) AS no_active_products FROM products WHERE is_active = TRUE
GROUP BY category HAVING COUNT(*) > 4;
131
132 -- C18. Top 3 most expensive products overall.
133 SELECT * FROM products ORDER BY price DESC LIMIT 3;
134
135 -- C19. Categories where average price is above 2,000.
136 SELECT category , AVG(price) AS avg_price FROM products GROUP BY category HAVING
AVG(price) > 2000;
137
138 -- --- C6: NULL Handling ---
139
140 -- C20. Products where avg_rating is missing.
141 SELECT * FROM products WHERE avg_rating IS NULL;

```

```

142
143 -- C21. product_name and rating; show 'New Launch' if avg_rating is NULL.
144 SELECT product_name, COALESCE(avg_rating, 'New Launch') AS rating FROM products;
145
146 -- --- C7: CASE WHEN ---
147
148 -- C22. price_tier column: Budget (<1000), Mid (<10000), Premium (>=10000).
149 SELECT
150     product_id,
151     product_name,
152     category,
153     brand,
154     price,
155     CASE
156         WHEN price < 1000 THEN 'Budget'
157         WHEN price < 10000 THEN 'Mid'
158         ELSE 'Premium'
159     END AS price_tier
160 FROM
161     products;
162
163 -- C23. Per category: total product count + count of Premium products (>=10000), using
164 SUM(CASE WHEN...).
165 SELECT
166     category,
167     COUNT(*) AS total_product_count,
168     SUM(CASE WHEN price >= 10000 THEN 1 ELSE 0 END) AS premium_product_count
169 FROM
170     products
171 GROUP BY category;
172
173 -- C24. Per category (only categories with >=3 products): total count, active count,
174 --      avg price, category_tier (Cheap <1500, Standard <10000, Luxury >=10000).
175 --      Sort by avg price descending.
176
177 SELECT
178     category,
179     COUNT(*) AS product_count,
180     SUM(CASE
181         WHEN is_active = TRUE THEN 1
182         ELSE 0
183     END) AS no_of_active_product,
184     AVG(price) AS avg_price,
185     CASE
186         WHEN AVG(price) < 1500 THEN 'Cheap'
187         WHEN AVG(price) < 10000 THEN 'Standard'
188         ELSE 'Luxury'
189     END AS category_tier
190 FROM
191     products
192 GROUP BY category
193 HAVING COUNT(*) >= 3
194 ORDER BY AVG(price) DESC;

```